

EDocs - Electronic Document Archiving System

Overview

EDocs is a comprehensive electronic document archiving system designed for government agencies and organizations to efficiently manage, store, and retrieve documents. Built with ASP.NET Framework 4.5.2 and MySQL database, this system provides a secure, scalable solution for document lifecycle management.

Features

- **Document Management**: Complete CRUD operations for documents with metadata
- **Advanced Search**: Full-text search and filtering by multiple criteria
- **User Management**: Role-based access control (Admin, Manager, User)
- **Category Hierarchy**: Flexible document classification system
- **Secure File Handling**: Encrypted file storage with decryption on demand
- **Messaging System**: Internal document-related communication between users
- **Audit Trail**: Complete tracking of document access and modifications
- **Responsive Design**: Modern Bootstrap 4 interface compatible with all devices
- **Multilingual Support**: Arabic interface with UTF-8 encoding

Technology Stack

- **Backend**: ASP.NET Framework 4.5.2, C#
- **Frontend**: HTML5, CSS3, JavaScript, Bootstrap 4
- **Database**: MySQL 5.7+ (migrated from Oracle)
- **ORM**: Entity Framework 6
- **Authentication**: ASP.NET Identity
- **File System**: Secure file storage with encryption/decryption
- **Search**: Custom LINQ-based search with pagination

Database Migration

This system was originally designed for Oracle database and has been successfully migrated to MySQL:

- Replaced Oracle.ManagedDataAccess with MySql.Data (8.0.33)
- Updated Entity Framework provider to MySql.Data.EntityFramework
- Modified all database connection strings and queries
- Updated configuration files (Web.config, packages.config, .csproj)
- Converted Oracle-specific SQL to MySQL syntax (e.g., "SELECT USER()" instead of "SELECT user FROM dual")

Installation & Setup

Prerequisites

- Visual Studio 2017 or later
- MySQL Server 5.7+
- .NET Framework 4.5.2
- Git

Steps

1. Clone the repository:

```
``bash
git clone https://github.com/your-username/edocs.git
cd edocs
``
```

2. Restore NuGet packages:

- The packages.config file is already configured for MySQL dependencies

3. Configure database connection:

- Update Web.config connection strings:

```
``xml

<add name="IdentityContext"
connectionString="server=YOUR_SERVER;userid=YOUR_USER;password=YOUR_PASSWORD;database=YOUR_DATABASE" providerName="MySql.Data.MySqlClient" />

<add name="OracleDbContext" providerName="MySql.Data.MySqlClient"
connectionString="server=localhost;userid=mysql_user;password=mysql_user_password;database=mysql_db" />

``
```

4. Create MySQL database:

```
``sql

CREATE DATABASE edocs CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

``
```

5. Run Entity Framework migrations (if applicable) or execute the database schema script

6. Build and run the solution in Visual Studio

Usage

User Roles

- ****Administrator****: Full access to all features including user and system management
- ****Manager****: Access to document management and reporting within assigned categories
- ****User****: Basic document viewing, searching, and messaging capabilities

Key Workflows

1. **Document Upload**: Navigate to Documents → Create to upload new documents with metadata
2. **Document Search**: Use the search functionality in the document listing page
3. **Document Retrieval**: Click on document details to view/download files
4. **Internal Messaging**: Use the DOCSSENT module to send/receive document-related messages
5. **Administration**: Manage users, roles, categories, and permissions from the admin panel

Security Features

- Role-based access control (RBAC)
- Input validation and sanitization
- Protection against CSRF attacks
- Secure password hashing with ASP.NET Identity
- Encrypted file storage for sensitive documents
- Session management and timeout
- SQL injection prevention through parameterized queries

Project Structure

...

/EDocs

- /Controllers - MVC Controllers (DOCUMENT, DOCSSENT, Account, etc.)
- /Models - Data models and ViewModels
- /Views - Razor views organized by controller
- /Content - CSS, JavaScript, and image assets
- /Scripts - Custom JavaScript files
- /App_Data - Application data (logs, etc.)
- /Properties - Assembly information

Web.config - Application configuration
packages.config - NuGet package dependencies
EDocs.csproj - Project file

...

API Endpoints (Key Controllers)

DOCUMENT Controller

- `GET /Document/Index` - Document listing with filtering
- `GET /Document/Details/{id}` - Document details view
- `GET /Document/Create` - Document upload form
- `POST /Document/Create` - Document upload handler
- `GET /Document/Edit/{id}` - Document edit form
- `POST /Document/Edit/{id}` - Document update handler
- `GET /Document/Delete/{id}` - Document delete confirmation
- `POST /Document/Delete/{id}` - Document deletion handler
- `POST /Document/GetList` - AJAX data source for DataTables

DOCSSENT Controller

- `GET /DOCSSENT/Index` - List of all sent/received messages
- `GET /DOCSSENT/Sentmasseges` - User's sent messages
- `GET /DOCSSENT/Recmassages` - User's received messages
- `GET /DOCSSENT/SearchResult` - Document-specific message search
- `POST /DOCSSENT/Create` - Send new message
- `POST /DOCSSENT/Reply` - Reply to existing message

Account Controller

- `GET /Account/Login` - Login page
- `POST /Account/Login` - Authentication handler

- `GET /Account/Register` - User registration
- `POST /Account/Register` - User creation
- `GET /Account/Logout` - Session termination

Configuration

Web.config Key Sections

- **Connection Strings**: MySQL database connections
- **appSettings**: Application-specific settings (paths, application name)
- **system.web**: ASP.NET configuration (compilation, authentication)
- **entityFramework**: Entity Framework MySQL provider configuration
- **system.data**: Database provider factories (MySQL Data Provider)
- **runtime**: Assembly binding redirects
- **log4net**: Logging configuration

appSettings Keys

- `webpages:Version` - ASP.NET Web Pages version
- `paths` - Backup directory path
- `applicationname` - Application display name (shown in header)
- `webpages:Enabled` - Web Pages feature toggle
- `ClientValidationEnabled` - Client-side validation
- `UnobtrusiveJavaScriptEnabled` - Unobtrusive jQuery validation

Customization

Changing Application Appearance

1. Modify CSS files in `/Content` directory
2. Update Bootstrap theme in `\_MainLayout.cshtml`
3. Change logo image in `/images` directory

4. Adjust color scheme in site.css

Adding Document Categories

1. Navigate to Administration → Manage Categories
2. Create parent and child categories as needed
3. Assign attributes to categories for document metadata

Configuring User Permissions

1. Navigate to Administration → Manage Roles
2. Create or modify roles (Admin, Manager, User)
3. Assign category permissions to roles via UserCategory management

Maintenance

Backup Procedures

1. Database backup: Use MySQL Workbench or mysqldump

```
```bash
mysqldump -u username -p edocs > edocs_backup.sql
```
```
2. File backup: Copy the `/content/uploads` directory structure
3. Configuration backup: Save Web.config and connection strings

Logs

- Application logs are configured via log4net in Web.config
- Log files are stored in the location specified in the log4net configuration
- Default log location: E:\logfile.txt (configurable)

Performance Optimization

- Enable browser caching for static content

- Consider implementing Redis for session storage in multi-server environments
- Optimize MySQL queries and add appropriate indexes
- Enable Entity Framework query caching where applicable
- Use CDN for external resources (jQuery, Bootstrap, etc.)

Troubleshooting

Common Issues

1. ****Database Connection Failures****

- Verify MySQL service is running
- Check connection strings in Web.config
- Confirm user has appropriate database privileges
- Ensure MySQL Connector/.NET is compatible with version

2. ****File Access/Permission Errors****

- Check IIS application pool identity has read/write access to upload directories
- Verify file encryption/decryption key is available (schema name + constant)
- Confirm temporary directories exist and are accessible

3. ****Authentication Problems****

- Clear browser cookies and cache
- Verify machineKey in Web.config is consistent across servers (if in farm)
- Check ASP.NET IIS registration (aspnet_regiis.exe)

4. ****Performance Degradation****

- Monitor MySQL slow query log
- Check for missing indexes on frequently queried columns
- Review Entity Framework generated SQL for efficiency
- Consider adding output caching for expensive operations

Contributing

1. Fork the repository
2. Create a feature branch (`git checkout -b feature/amazing-feature`)
3. Commit your changes (`git commit -m 'Add amazing feature'`)
4. Push to the branch (`git push origin feature/amazing-feature`)
5. Open a Pull Request

Please ensure your code follows the existing coding standards and includes appropriate comments.

License

This project is licensed under the MIT License - see the LICENSE file for details.

Contact

For questions, support, or customization requests, please contact:

****Mahmoud Shaheen**** - Lead Developer

Email: mahmoudshaheensy@gmail.com

Acknowledgments

- MySQL Connector/NET team for the excellent MySQL ADO.NET provider
- Bootstrap team for the responsive frontend framework
- JetBrains for development tools support
- The open-source community for various libraries and tools used

Last updated: May 2026

Version: 1.0.0